

EE 5325 Course Project: Robot Arm Control

Spring 2026

Name	ID	Contribution	Peer Evaluation (0-100)
Akhilesh	1002248929	Problems 7, 8a, 8b	100
Bryan	1001034671	Problems 4, 5, 6	100
Jose	1001778907	Problems 1, 2, 3	100

Part 1: Modeling

Problem 1

- Solution:**

The dynamics of a 2-link planar robot arm were derived using the Euler-Lagrange formulation. The positions of the masses were first expressed using forward kinematics, and their velocities were obtained by differentiation.

The kinetic energy was computed assuming point masses at the link ends, while the potential energy was derived based on gravitational effects. The Lagrangian was then formed as $L = T - V$.

Applying the Euler-Lagrange equations yielded the nonlinear robot dynamics in standard form:

$$M(q)\ddot{q} + N(q, \dot{q}) = \tau$$

where $M(q)$ is the inertia matrix and $N(q, \dot{q})$ contains Coriolis, centrifugal, and gravitational terms. The resulting equations capture the coupled and nonlinear behavior of the two-link manipulator.

Part I: Modeling

Problem 1: Robot Dynamics

∴ Masses are concentrated at the end of the links:

position of mass 1, m_1 :

$$p_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} a_1 \cos \theta_1 \\ a_1 \sin \theta_1 \end{bmatrix}$$

position of mass 2, m_2 :

$$p_2 = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) \\ a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2) \end{bmatrix}$$

velocity of each mass:

$$\dot{p}_1 = \begin{bmatrix} -a_1 \sin \theta_1 \dot{\theta}_1 \\ a_1 \cos \theta_1 \dot{\theta}_1 \end{bmatrix}$$

$$\therefore v_1^2 = \dot{p}_1^T \dot{p}_1 = a_1^2 \dot{\theta}_1^2$$

$$\dot{p}_2 = \begin{bmatrix} -a_1 \sin \theta_1 \dot{\theta}_1 - a_2 \sin(\theta_1 + \theta_2)(\dot{\theta}_1 + \dot{\theta}_2) \\ a_1 \cos \theta_1 \dot{\theta}_1 + a_2 \cos(\theta_1 + \theta_2)(\dot{\theta}_1 + \dot{\theta}_2) \end{bmatrix}$$

$$\therefore v_2^2 = \dot{p}_2^T \dot{p}_2$$

$$\Rightarrow v_2^2 = a_1^2 \dot{\theta}_1^2 + a_2^2 (\dot{\theta}_1 + \dot{\theta}_2)^2 + 2a_1 a_2 \cos \theta_2 \dot{\theta}_1 (\dot{\theta}_1 + \dot{\theta}_2)$$

$$\Rightarrow v_2^2 = (a_1^2 + a_2^2 + 2a_1 a_2 \cos \theta_2) \dot{\theta}_1^2 + a_2^2 \dot{\theta}_2^2 + 2(a_1^2 + a_1 a_2 \cos \theta_2) \dot{\theta}_1 \dot{\theta}_2$$

$\therefore$ Masses are treated as point masses, only translational Kinetic Energy is included.

$$T = \frac{1}{2} m_1 v_1^2 + \frac{1}{2} m_2 v_2^2$$

$$\Rightarrow T = \frac{1}{2} m_1 a_1^2 \dot{\theta}_1^2 + \frac{1}{2} m_2 [(a_1^2 + a_2^2 + 2a_1 a_2 \cos \theta_2) \dot{\theta}_1^2 + a_2^2 \dot{\theta}_2^2 + 2(a_1^2 + a_1 a_2 \cos \theta_2) \dot{\theta}_1 \dot{\theta}_2]$$

$$\Rightarrow T = \frac{1}{2} [(m_1 + m_2) a_1^2 + m_2 a_2^2 + 2m_2 a_1 a_2 \cos \theta_2] \dot{\theta}_1^2 + \frac{1}{2} m_2 a_2^2 \dot{\theta}_2^2 + m_2 (a_1^2 + a_1 a_2 \cos \theta_2) \dot{\theta}_1 \dot{\theta}_2$$

Taking the upward vertical direction as +ve y.

Potential Energy, $V = mgy$

$$\Rightarrow V = m_1 g y_1 + m_2 g y_2$$

$$\Rightarrow V = (m_1 + m_2) g a_1 \sin \theta_1 + m_2 g a_2 \sin(\theta_1 + \theta_2)$$

Lagrangian, $L = T - V$

Using Euler-Lagrangian Eqn. for each joint:

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}_i} \right) - \frac{\partial L}{\partial \theta_i} = \tau_i \quad ; i = 1, 2$$

$$\text{Let } h = m_2 a_1 a_2 \sin \theta_2$$

From Kinetic Energy, inertia matrix is,

$$M(q) = \begin{bmatrix} M_{11} & M_{12} \\ M_{21} & M_{22} \end{bmatrix}$$

$$\Rightarrow M(q) = \begin{bmatrix} (m_1 + m_2) a_1^2 + m_2 a_2^2 + 2m_2 a_1 a_2 \cos \theta_2 & m_2 a_1^2 + m_2 a_1 a_2 \cos \theta_2 \\ m_2 a_1^2 + m_2 a_1 a_2 \cos \theta_2 & m_2 a_2^2 \end{bmatrix}$$

$$\Rightarrow \text{Gravity Vector,}$$

$$G(q) = \begin{bmatrix} \frac{\partial V}{\partial \theta_1} \\ \frac{\partial V}{\partial \theta_2} \end{bmatrix} = \begin{bmatrix} (m_1 + m_2)g a_1 \cos \theta_1 + m_2 a_2 \cos(\theta_1 + \theta_2) \\ m_2 g a_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

$$\Rightarrow \text{Coriolis \& Centrifugal terms,}$$

$$C(q, \dot{q})\dot{q} = \begin{bmatrix} -h(2\dot{\theta}_1\dot{\theta}_2 + \dot{\theta}_2^2) \\ h\dot{\theta}_1^2 \end{bmatrix}$$

Final Robot Dynamics :

$$\therefore M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

$$(\text{OR}) M(q)\ddot{q} + N(q, \dot{q}) = \tau$$

with,

$$N(q, \dot{q}) = \begin{bmatrix} -h(2\dot{\theta}_1\dot{\theta}_2 + \dot{\theta}_2^2) + (m_1 + m_2)g a_1 \cos \theta_1 + m_2 g a_2 \cos(\theta_1 + \theta_2) \\ h\dot{\theta}_1^2 + m_2 g a_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} (m_1 + m_2)a_1^2 + m_2 a_2^2 + 2m_2 a_1 a_2 \cos \theta_2 \end{bmatrix} \ddot{\theta}_1 + \begin{bmatrix} m_2 a_2^2 + m_2 a_1 a_2 \cos \theta_2 \end{bmatrix} \ddot{\theta}_2 - m_2 a_1 a_2 \sin \theta_2 (2\dot{\theta}_1\dot{\theta}_2 + \dot{\theta}_2^2) + (m_1 + m_2)g a_1 \cos \theta_1 + m_2 g a_2 \cos(\theta_1 + \theta_2) = \tau_1$$

&

$$\begin{bmatrix} m_2 a_2^2 + m_2 a_1 a_2 \cos \theta_2 \end{bmatrix} \ddot{\theta}_1 + m_2 a_1^2 \ddot{\theta}_2 + m_2 a_1 a_2 \sin \theta_2 \dot{\theta}_1^2 + m_2 g a_2 \cos(\theta_1 + \theta_2) = \tau_2 //$$

Problem 2

• Solution:

The nonlinear robot dynamics were converted into a state-space representation. The state vector was defined as:

$$x = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^T$$

Using the dynamic equation $M(q)\ddot{q} + N(q, \dot{q}) = \tau$, the accelerations were expressed as:

$$\ddot{q} = M^{-1}(q)(\tau - N(q, \dot{q}))$$

This resulted in the full nonlinear state-space model:

$$\dot{x} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \ddot{\theta}_1 \\ \ddot{\theta}_2 \end{bmatrix}$$

Which describes the system dynamics in a form suitable for simulation and control design.

Problem 2: State Space Form:

$$\text{Given } x = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}, \quad u = \tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

$$\Rightarrow x_1 = \theta_1, \quad x_2 = \theta_2, \quad x_3 = \dot{\theta}_1, \quad x_4 = \dot{\theta}_2$$

Then,

$$\dot{x}_1 = x_3$$

$$\dot{x}_2 = x_4$$

$$\text{From } M(q)\ddot{q} = \tau - N(q, \dot{q})$$

$$\Rightarrow \ddot{q} = M^{-1}(q)(\tau - N(q, \dot{q}))$$

$$\Rightarrow \begin{bmatrix} \ddot{x}_3 \\ \ddot{x}_4 \end{bmatrix} = M^{-1}(x_1, x_2) \left(\begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} - N(x_1, x_2, x_3, x_4) \right)$$

$\therefore$ full State Space Model:

$$\dot{x} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} x_3 \\ x_4 \\ [M^{-1}(q)(\tau - N(q, \dot{q}))]_1 \\ [M^{-1}(q)(\tau - N(q, \dot{q}))]_2 \end{bmatrix}$$

$$\text{where } q = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \dot{q} = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix}$$

Part 2: Open-Loop Simulation

Problem 3

- **Solution:**

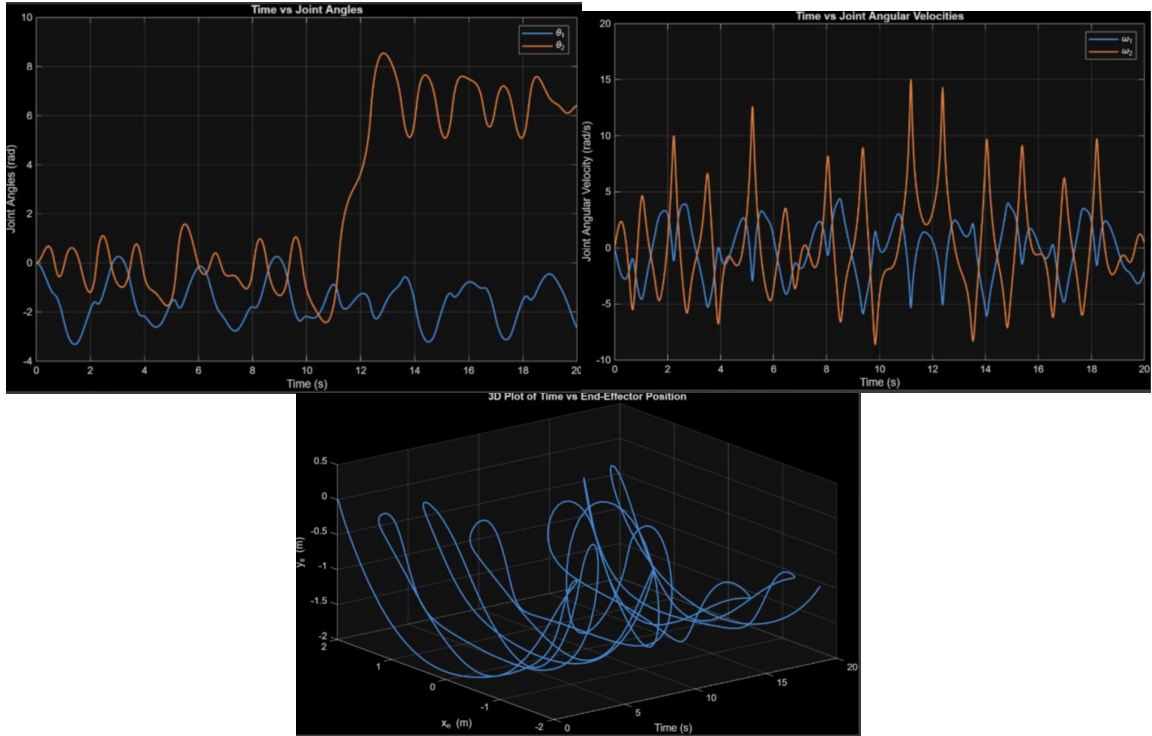
The given MATLAB code solves the motion of a 2-link robotic arm using numerical integration. Because the system is nonlinear and coupled, there is no closed-form analytical solution. Instead, the solution is obtained numerically as a function of time. The output of the solver is the matrix $x(t)$, which contains $\theta_1(t)$ and $\theta_2(t)$ (joint angles) and $\dot{\theta}_1(t)$ and $\dot{\theta}_2(t)$ (joint angular velocities). These values represent the system's complete solution over the time span.

The code also computes the end-effector position:

- $x_e = a_1 \cos(\theta_1) + a_2 \cos(\theta_1 + \theta_2)$
- $y_e = a_1 \sin(\theta_1) + a_2 \sin(\theta_1 + \theta_2)$

Three plots are generated: 1. Joint angles vs time 2. Angular velocities vs time 3. End-effector position vs time (3D plot). Overall, the solution describes how the arm moves over time under the applied torques, and it is obtained numerically rather than a simple equation.

- **Simulation Results:**



MATLAB Codes: Part_II_Open_Loop_Simulation_Problem_3.m

Part 3: Closed-Loop Control

Problem 4

- **Solution:**

For the 2-link planar elbow robot arm, the closed-loop dynamics are based on the nonlinear model

Nonlinear Dynamics

$$M(q)\ddot{q} + N(q, \dot{q}) = \tau$$

State Variables

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \quad \dot{q} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}, \quad \tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

System Parameters

$$m_1 = m_2 = 1 \text{ kg}, \quad a_1 = a_2 = 1 \text{ m}, \quad g = 9.8 \text{ m/s}^2$$

The desired joint trajectories are:

Desired Joint Trajectories

$$q_d(t) = \begin{bmatrix} 0.1 \sin(2\pi t) \\ 0.1 \cos(2\pi t) \end{bmatrix}$$

Velocity

$$\dot{q}_d(t) = \begin{bmatrix} 0.2\pi \cos(2\pi t) \\ -0.2\pi \sin(2\pi t) \end{bmatrix}$$

Initial State

$$q(0) = \begin{bmatrix} \frac{\pi}{3} \\ \frac{\pi}{3} \end{bmatrix}, \quad \dot{q}(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

PD-Gravity Controller

$$\tau = K_p(q_d - q) + K_v(\dot{q}_d - \dot{q}) + G(q)$$

$$K_p = \begin{bmatrix} 2500 & 0 \\ 0 & 2500 \end{bmatrix}, \quad K_v = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix}$$

The gravity compensation term is

Gravity Term

$$G(q) = \begin{bmatrix} (m_1 + m_2)ga_1 \cos \theta_1 + m_2ga_2 \cos(\theta_1 + \theta_2) \\ m_2ga_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

Numerical Form

$$G(q) = \begin{bmatrix} 19.6 \cos \theta_1 + 9.8 \cos(\theta_1 + \theta_2) \\ 9.8 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

Control Input

$$\tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} = K_p(q_d - q) + K_v(\dot{q}_d - \dot{q}) + G(q)$$

$$\tau_1 = 2500(q_{d1} - \theta_1) + 100(\dot{q}_{d1} - \dot{\theta}_1) + 19.6 \cos \theta_1 + 9.8 \cos(\theta_1 + \theta_2)$$

$$\tau_2 = 2500(q_{d2} - \theta_2) + 100(\dot{q}_{d2} - \dot{\theta}_2) + 9.8 \cos(\theta_1 + \theta_2)$$

Forward Kinematics

$$x = a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2)$$

$$y = a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2)$$

With $a_1 = a_2 = 1$:

$$x = \cos \theta_1 + \cos(\theta_1 + \theta_2)$$

$$y = \sin \theta_1 + \sin(\theta_1 + \theta_2)$$

The simulation results show that the controller drives the manipulator quickly from the initial condition to the desired periodic trajectories with very small tracking error after a short transient period.

Discussion: Limitations in the Control Design

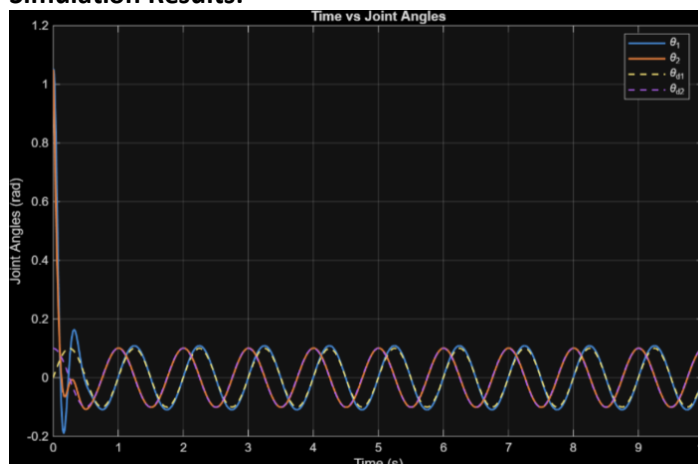
The main limitation of the PD-gravity controller is that it compensates only for gravity and does not explicitly cancel the Coriolis and centrifugal terms in the nonlinear robot dynamics. Because of this, its tracking performance may degrade when the robot moves at higher speeds or follows more aggressive trajectories.

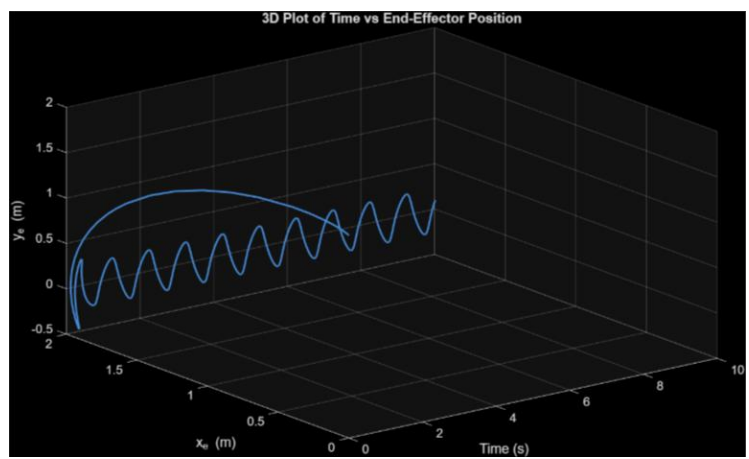
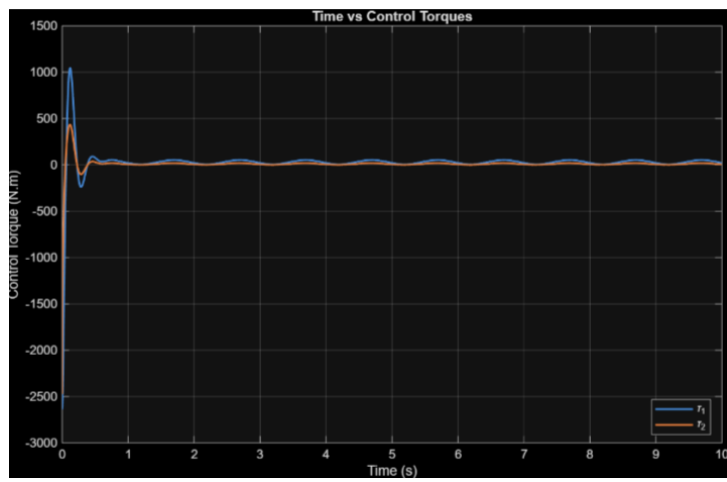
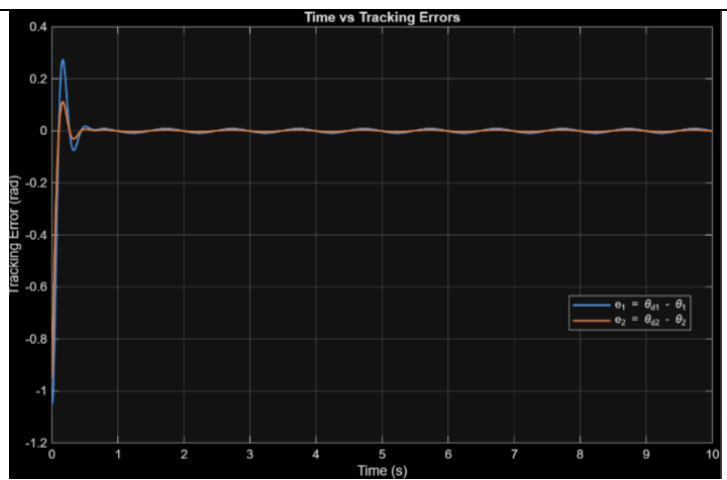
Another limitation is that small tracking errors are achieved by using very high feedback gains. This results in a very large initial control torque when the initial error is large. In a real system, such large torque demand may exceed actuator limits, cause saturation, and reduce practical performance.

The controller also assumes that the robot parameters, such as masses, link lengths, and gravity-related terms, are known. If there are modeling errors, parameter uncertainty, or external disturbances, the gravity compensation term will not be exact, and the tracking error can increase.

Therefore, although the PD-gravity controller is simple and provides good tracking for this case, it is not fully robust to dynamic coupling, uncertainty, and disturbance. More advanced controllers such as computed-torque control, adaptive control, or robust control can provide better performance under such conditions.

- **Simulation Results:**





- **Discussion:**

The results show that the PD-gravity controller performs well for the 2-link robot arm. The joint angles quickly move from the initial condition and begin closely following the desired sinusoidal trajectories, while the tracking errors decrease rapidly and remain very small. The 3D end-effector plot also shows that the motion becomes smooth and repeatable once the system stabilizes. However, the large initial transient indicates a high control effort, which may not be ideal for real systems and suggests that more advanced controllers could improve performance.

- **MATLAB Codes:**

Part_III_Closed_Loop_Control_Problem_4.m

Problem 5

- **Solution:**

To overcome the limitation of the PD-gravity controller in Problem 4, a computed-torque controller is designed. The limitation of the PD-gravity controller is that it compensates only for gravity and does not explicitly cancel the Coriolis, centrifugal, and inertia coupling terms of the robot dynamics. Because of this, its tracking performance may degrade when the manipulator's motion becomes faster or when nonlinear coupling effects become significant.

The nonlinear dynamics of the 2-link planar elbow robot arm are:

Nonlinear Dynamics

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

State and Input

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \quad \tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

System Parameters

$$m_1 = m_2 = 1 \text{ kg}, \quad a_1 = a_2 = 1 \text{ m}, \quad g = 9.8 \text{ m/s}^2$$

Desired Joint Trajectories

$$q_d(t) = \begin{bmatrix} 0.1 \sin(2\pi t) \\ 0.1 \cos(2\pi t) \end{bmatrix}$$

First Derivative

$$\dot{q}_d(t) = \begin{bmatrix} 0.2\pi \cos(2\pi t) \\ -0.2\pi \sin(2\pi t) \end{bmatrix}$$

Second Derivative

$$\ddot{q}_d(t) = \begin{bmatrix} -0.4\pi^2 \sin(2\pi t) \\ -0.4\pi^2 \cos(2\pi t) \end{bmatrix}$$

The initial state is.

Computed-Torque Controller Design

Initial State

$$q(0) = \begin{bmatrix} \frac{\pi}{3} \\ \frac{\pi}{3} \end{bmatrix}, \quad \dot{q}(0) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Tracking Errors

$$e = q_d - q, \quad \dot{e} = \dot{q}_d - \dot{q}$$

Control Law

$$\tau = M(q) \left[\ddot{q}_d + K_v(\dot{q}_d - \dot{q}) + K_p(q_d - q) \right] + C(q, \dot{q})\dot{q} + G(q)$$

Gain Matrices

$$K_p = \begin{bmatrix} 2500 & 0 \\ 0 & 2500 \end{bmatrix}, \quad K_v = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix}$$

Define

$$v = \ddot{q}_d + K_v(\dot{q}_d - \dot{q}) + K_p(q_d - q)$$

Then,

$$\tau = M(q)v + C(q, \dot{q})\dot{q} + G(q)$$

Substituting into the robot dynamics:

$$M(q)\ddot{q} = M(q)v \Rightarrow \ddot{q} = v$$

So the closed-loop error dynamics become:

$$\ddot{e} + K_v\dot{e} + K_p e = 0$$

This is a linear second-order stable error system. Hence, with positive definite gain matrices K_p and K_v , the tracking error converges to zero.

The simulation was carried out in MATLAB using ode23 over the interval $0 \leq t \leq 10$ s. The end-effector position was computed from the forward kinematics

Closed-Loop Behavior and End-Effector Position

Error Dynamics

$$\ddot{e} + K_v \dot{e} + K_p e = 0$$

Since this is a linear second-order stable system with positive definite K_p and K_v ,

$$e(t) \rightarrow 0$$

which means the tracking error converges to zero.

Simulation

$$0 \leq t \leq 10 \text{ s}$$

Forward Kinematics

$$x_e = a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2)$$

$$y_e = a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2)$$

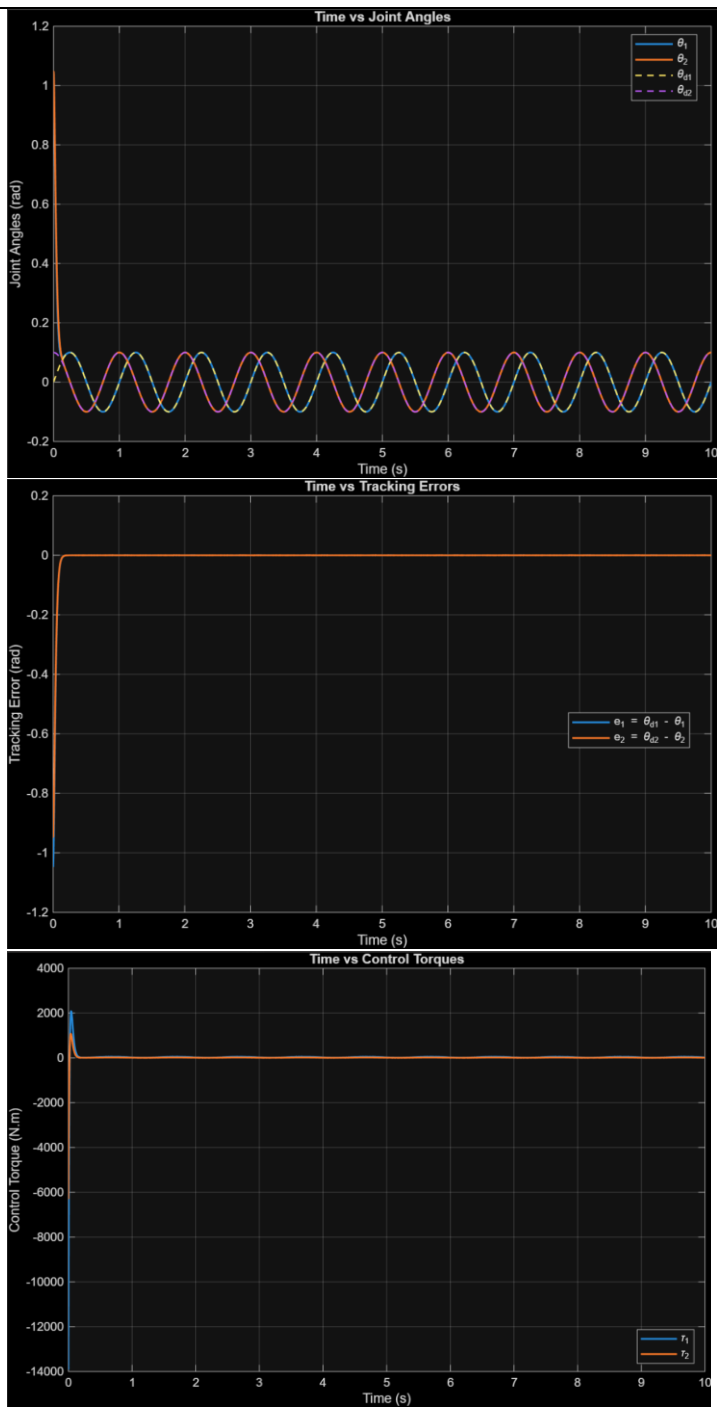
With $a_1 = a_2 = 1$,

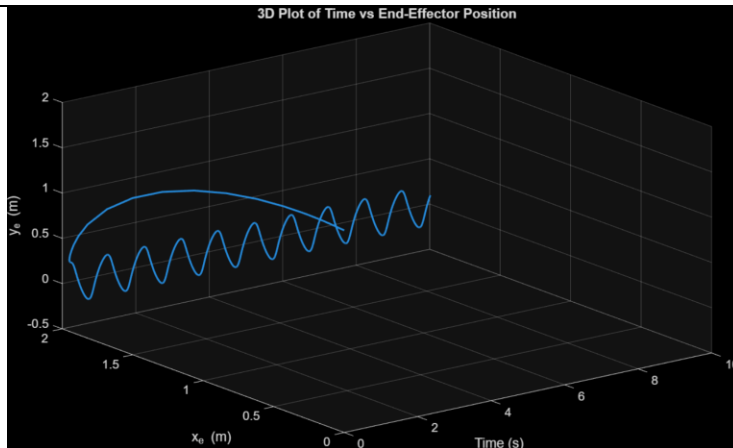
$$x_e = \cos \theta_1 + \cos(\theta_1 + \theta_2)$$

$$y_e = \sin \theta_1 + \sin(\theta_1 + \theta_2)$$

- **Simulation Results:**

The simulation results show that the joint angles quickly converge to the desired sinusoidal trajectories, and the tracking errors approach zero almost immediately after the initial transient. This confirms that the computed-torque controller achieves very accurate tracking by compensating the full nonlinear robot dynamics.





- **MATLAB Codes:** Part_III_Closed_Loop_Control_Problem_5.m
- **Discussion:**

The computed-torque controller achieves excellent tracking with near zero steady-state error and smooth, repeatable motion. While it requires higher initial torque and accurate system modeling, it outperforms the PD-gravity controller.

Problem 6

- **Solution:**

In Problem 6, constant disturbance torques are added to both joints of the robot arm. The disturbed robot dynamics are written as

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau + \tau_d$$

where the disturbance vector is

$$\tau_d = \begin{bmatrix} 5 \\ 5 \end{bmatrix} \text{ Nm}$$

The objective is to design a controller such that the tracking error converges to zero even in the presence of this constant disturbance. In the MATLAB code, this disturbance is implemented as `params.taud = [5; 5]`.

A controller without integral action may not remove the steady-state error caused by constant disturbance torques. Therefore, to achieve zero steady-state tracking error, a **computed-torque controller with integral action** is designed. This controller combines nonlinear dynamic compensation with proportional, derivative, and integral feedback.

The desired joint trajectories are the same as in Problem 4:

$$q_{d1} = 0.1 \sin(2\pi t)$$

$$q_{d2} = 0.1 \cos(2\pi t)$$

Their first derivatives are

$$\dot{q}_{d1} = 0.2\pi \cos(2\pi t)$$

$$\dot{q}_{d2} = -0.2\pi \sin(2\pi t)$$

and their second derivatives are

$$\ddot{q}_{d1} = -0.4\pi^2 \sin(2\pi t)$$

$$\ddot{q}_{d2} = -0.4\pi^2 \cos(2\pi t)$$

The tracking error and its derivative are defined as

$$e = q_d - q$$

$$\dot{e} = \dot{q}_d - \dot{q}$$

To introduce integral action, the integral of the tracking error is defined as

$$\dot{\eta} = e$$

where

$$\eta = \begin{bmatrix} \eta_1 \\ \eta_2 \end{bmatrix}$$

is the integral error vector.

The controller is designed using the computed-torque approach. This choice is made because the robot arm is a nonlinear and coupled system, so simple linear joint feedback is not enough for accurate tracking. The computed-torque method uses the robot model to compensate for the nonlinear dynamics directly, while the feedback terms shape the tracking behavior.

Define

$$v = \ddot{q}_d + K_v \dot{e} + K_p e + K_i \eta$$

Then the control law is selected as

$$\tau = M(q)v + C(q, \dot{q})\dot{q} + G(q)$$

or equivalently,

$$\tau = M(q)[\ddot{q}_d + K_v\dot{e} + K_p e + K_i\eta] + C(q, \dot{q})\dot{q} + G(q)$$

This control law was designed in this form for the following reasons.

The term $\ddot{q}_d$ provides feedforward acceleration so that the controller anticipates the desired motion.

The proportional term $K_p e$ reduces position error.

The derivative term $K_v \dot{e}$ adds damping and reduces oscillation.

The integral term $K_i \eta$ is included to reject constant disturbance torques and eliminate steady-state tracking error.

The terms $M(q)$, $C(q, \dot{q})\dot{q}$, and $G(q)$ are included so that the controller compensates for the nonlinear robot dynamics directly. If no disturbance were present, this controller would behave like a nonlinear PID tracking law. Once the constant disturbance is added, the integral term gradually builds the extra control effort needed to cancel that disturbance.

In the MATLAB code, the gain matrices are chosen as

$$K_p = \text{diag}(2500, 2500)$$

$$K_v = \text{diag}(100, 100)$$

$$K_i = \text{diag}(300, 300)$$

These are implemented as `params.Kp`, `params.Kv`, and `params.Ki` in the code.

The disturbed plant dynamics used in the MATLAB simulation are

$$\ddot{q} = M(q)^{-1}[\tau + \tau_d - C(q, \dot{q})\dot{q} - G(q)]$$

This shows clearly that the plant receives both the control torque and the disturbance torque. The controller itself does not directly know the disturbance; instead, the integral action compensates for it over time. This exact structure appears in the ODE implementation, where the plant is simulated using `ddq = M \ (tau + params.taud - Cvec - G)`.

For this problem, the robot parameters used in simulation are

$$m_1 = m_2 = 1 \text{ kg}$$

$$a_1 = a_2 = 1 \text{ m}$$

$$g = 9.8 \text{ m/s}^2$$

The augmented state vector is

$$x = [\theta_1 \quad \theta_2 \quad \dot{\theta}_1 \quad \dot{\theta}_2 \quad \eta_1 \quad \eta_2]^T$$

with initial condition

$$[\theta_1(0) \quad \theta_2(0) \quad \dot{\theta}_1(0) \quad \dot{\theta}_2(0) \quad \eta_1(0) \quad \eta_2(0)]^T = [\pi/3 \quad \pi/3 \quad 0 \quad 1 \quad 0 \quad 0]^T$$

In the MATLAB code, this is implemented as

$$x_0 = [\pi/3 \quad \pi/3 \quad 0 \quad 1 \quad 0 \quad 0]^T$$

The end-effector position is computed from the forward kinematics

$$x_e = a_1 \cos \theta_1 + a_2 \cos (\theta_1 + \theta_2)$$

$$y_e = a_1 \sin \theta_1 + a_2 \sin (\theta_1 + \theta_2)$$

Since $a_1 = a_2 = 1$, this becomes

$$x_e = \cos \theta_1 + \cos (\theta_1 + \theta_2)$$

$$y_e = \sin \theta_1 + \sin (\theta_1 + \theta_2)$$

These expressions are used to generate the 3D plot of time versus end-effector position.

Conclusion

For Problem 6, an integral computed-torque controller was designed for the 2-link planar elbow robot arm in the presence of constant disturbance torques. The controller combines nonlinear dynamic compensation with proportional, derivative, and integral error feedback. The integral term was added specifically because constant disturbances can create steady-state tracking error if only proportional and derivative feedback are used.

The final controller is

$$\tau = M(q)[\ddot{q}_d + K_v \dot{e} + K_p e + K_i \eta] + C(q, \dot{q})\dot{q} + G(q)$$

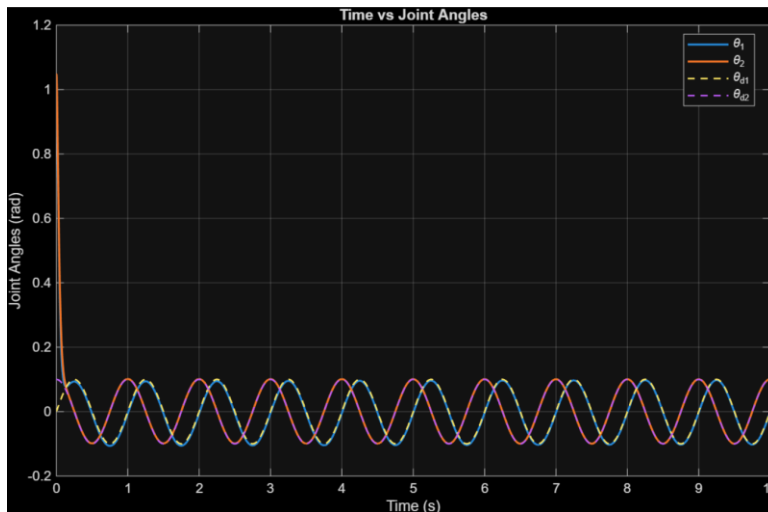
with disturbance acting through

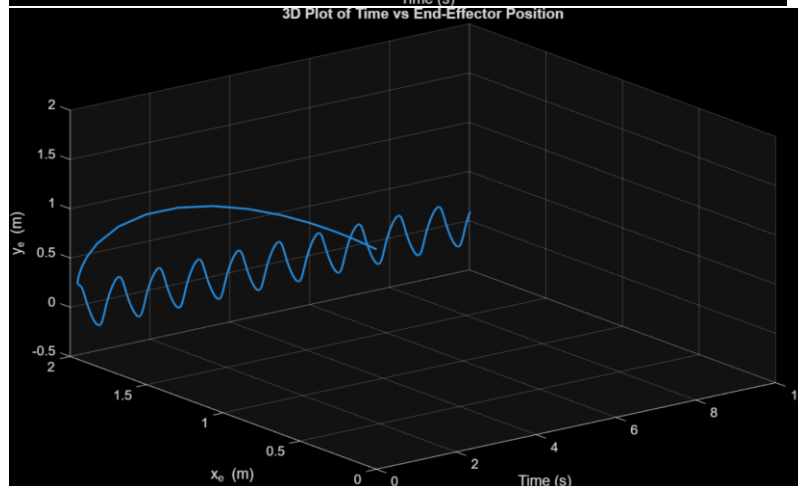
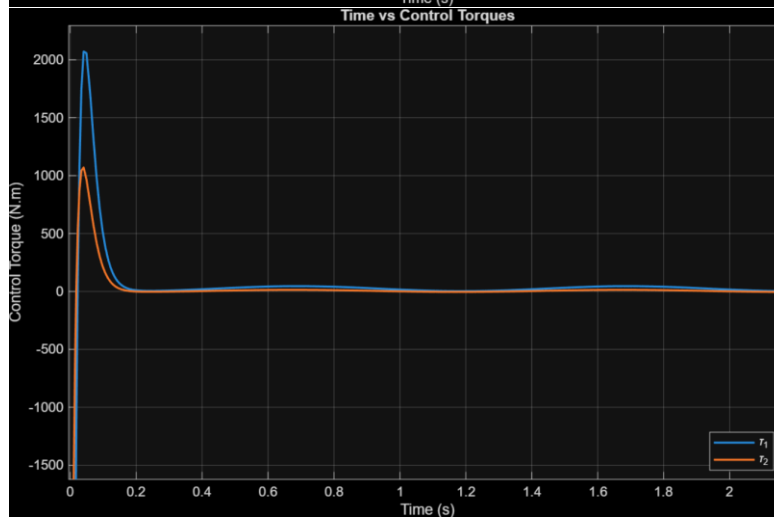
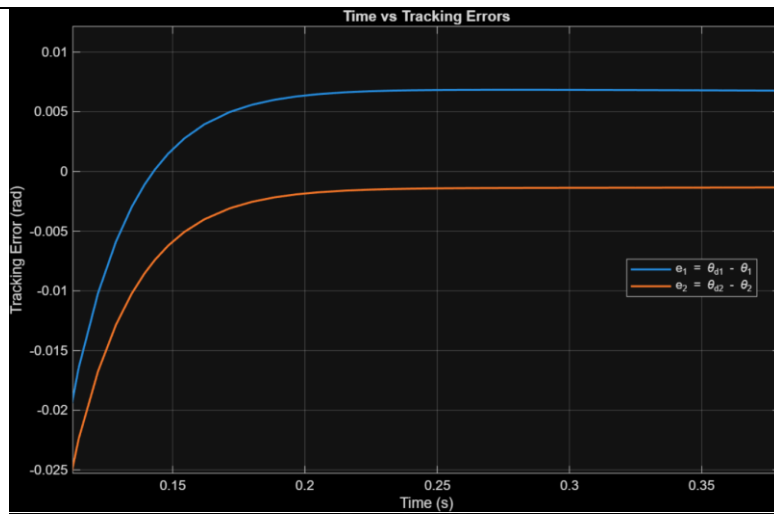
$$\tau_d = \begin{bmatrix} 5 \\ 5 \end{bmatrix} \text{ Nm}$$

The simulation results show that the controller successfully rejects the constant disturbance and drives the tracking error to zero. Therefore, the integral computed-torque controller provides accurate trajectory tracking even in the presence of constant external torque disturbances.

- **Simulation Results:**

The simulation results show that the integral computed-torque controller performs very well in the presence of the constant disturbance torque. From the joint-angle plot, both θ_1 and θ_2 quickly converge to their desired sinusoidal trajectories after a short transient. The tracking-error plot confirms that both errors decrease rapidly to nearly zero and remain very small throughout the rest of the simulation. This shows that the integral term successfully removes the steady-state error caused by the constant disturbance. The control-torque plot shows a large initial torque spike, which is expected because the controller must correct the initial condition error and reject the disturbance at the same time. After the transient, the torques settle into smooth periodic signals with much smaller magnitude. The 3D end-effector plot also shows an initial adjustment followed by smooth and periodic motion, indicating stable tracking behavior. Overall, the results confirm that the integral computed-torque controller successfully compensates for the constant disturbance and achieves accurate trajectory tracking, although the initial control effort is quite large.





- MATLAB Codes:**

Part_III_Closed_Loop_Control_Problem_6.m

- **Discussion:**

The integral computed-torque controller provides accurate tracking even with a constant disturbance torque applied to the system. The joint angles quickly converge to the desired sinusoidal trajectories, and the tracking errors decrease to zero, showing that the integral action successfully removes steady-state error. The control torque shows a large initial spike, followed by smaller periodic values once the system stabilizes and compensates for the disturbance. Overall, the controller achieves strong disturbance rejection and precise tracking, although the high initial control effort may be challenging for real-world systems.

Problem 7

- **Solution:**

In this problem, the 2-link planar elbow robot arm is controlled using a **PID computed-torque controller** while enforcing actuator torque limits of

$$-15 \leq \tau_1 \leq 15 \text{ Nm}, -15 \leq \tau_2 \leq 15 \text{ Nm}$$

The purpose of this design is to examine how a nonlinear model-based tracking controller performs when the actuators are unable to supply unlimited torque.

The robot dynamics are given by

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

where

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

For this robot, the inertia matrix, Coriolis/centrifugal vector, and gravity vector are implemented in MATLAB as

$$M(q) = \begin{bmatrix} (m_1 + m_2)a_1^2 + m_2a_2^2 + 2m_2a_1a_2\cos \theta_2 & m_2a_2^2 + m_2a_1a_2\cos \theta_2 \\ m_2a_2^2 + m_2a_1a_2\cos \theta_2 & m_2a_2^2 \end{bmatrix}$$

$$C(q, \dot{q})\dot{q} = \begin{bmatrix} -h(2\dot{\theta}_1\dot{\theta}_2 + \dot{\theta}_2^2) \\ h\dot{\theta}_1^2 \end{bmatrix}$$

where

$$h = m_2a_1a_2\sin \theta_2$$

and

$$G(q) = \begin{bmatrix} (m_1 + m_2)ga_1 \cos \theta_1 + m_2ga_2 \cos (\theta_1 + \theta_2) \\ m_2ga_2 \cos (\theta_1 + \theta_2) \end{bmatrix}$$

The robot parameters used in the simulation are

$$m_1 = m_2 = 1 \text{ kg}, a_1 = a_2 = 1 \text{ m}, g = 9.8 \text{ m/s}^2$$

The desired trajectories are the same as in Problem 4:

$$q_{d1} = 0.1 \sin (2\pi t), q_{d2} = 0.1 \cos (2\pi t)$$

with first derivatives

$$\dot{q}_{d1} = 0.2\pi \cos (2\pi t), \dot{q}_{d2} = -0.2\pi \sin (2\pi t)$$

and second derivatives

$$\ddot{q}_{d1} = -0.4\pi^2 \sin (2\pi t), \ddot{q}_{d2} = -0.4\pi^2 \cos (2\pi t)$$

The tracking errors are defined as

$$e = q_d - q$$

$$\dot{e} = \dot{q}_d - \dot{q}$$

To introduce integral action, the integral error state is defined as

$$\dot{\varepsilon} = e$$

where

$$\varepsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \end{bmatrix}$$

This gives the augmented state vector

$$x = [\theta_1 \quad \theta_2 \quad \dot{\theta}_1 \quad \dot{\theta}_2 \quad \varepsilon_1 \quad \varepsilon_2]^T$$

with the initial condition

$$x(0) = [0 \quad 0 \quad 1 \quad 2 \quad 0 \quad 0]^T$$

The controller is designed using the **computed-torque approach** because the robot arm is a nonlinear and coupled mechanical system. A simple linear PID controller acting directly on the joints would not account for the inertia coupling, Coriolis effects, or gravity terms. Therefore, the robot model is included explicitly in the control law so that the controller

can compensate for the nonlinear dynamics while imposing desired tracking behavior through PID feedback.

The commanded control torque is chosen as

$$\tau_c = M(q)[\ddot{q}_d + K_v \dot{e} + K_p e + K_i \varepsilon] + C(q, \dot{q})\dot{q} + G(q)$$

The gain matrices used in the MATLAB code are

$$K_p = \text{diag}(2500, 2500), K_v = \text{diag}(100, 100), K_i = \text{diag}(200, 200)$$

This design was chosen for the following reasons:

- I. The term $\ddot{q}_d$ provides feedforward acceleration so that the controller anticipates the desired motion rather than reacting to error alone.
- II. The proportional term $K_p e$ is used to reduce position error and force the joints toward the desired angles.
- III. The derivative term $K_v \dot{e}$ adds damping and reduces oscillatory behavior.
- IV. The integral term $K_i \varepsilon$ accumulates the tracking error over time and helps eliminate persistent bias in the response.

The multiplication by $M(q)$, together with the addition of $C(q, \dot{q})\dot{q}$ and $G(q)$, is what makes this a computed-torque controller. These terms are included so that the controller compensates for the nonlinear manipulator dynamics directly. If no torque saturation were present and the model were exact, this control law would make the tracking error behave approximately like a decoupled PID system. That is the main reason it was designed in this form.

Saturation design

Although τ_c is the ideal torque required for tracking, the actuator cannot apply torque outside the range $[-15, 15]$ Nm. Therefore, the actual control input to the robot is obtained by saturating the commanded torque element-wise:

$$\tau = \text{sat}(\tau_c)$$

with

$$\tau_i = \begin{cases} 15, & \tau_{c,i} > 15 \\ \tau_{c,i}, & -15 \leq \tau_{c,i} \leq 15 \\ -15, & \tau_{c,i} < -15 \end{cases} i = 1, 2$$

This saturation step is necessary because the actuator limits are physical constraints of the system. The controller may request a torque greater than the motor can provide, but the plant can only receive the bounded torque. For this reason, the actual plant dynamics are simulated using the saturated torque τ , not the ideal torque τ_c .

The plant dynamics used in the MATLAB code are therefore

$$\ddot{q} = M(q)^{-1}[\tau - C(q, \dot{q})\dot{q} - G(q)]$$

and the integral error state evolves as

$$\dot{\varepsilon} = e$$

Why the controller was designed this way

The controller was designed in two layers.

First, a nonlinear computed-torque law was used because the robot dynamics are nonlinear and coupled. This gives a physically meaningful controller structure and improves tracking compared to a plain PID law.

Second, a saturation block was added because the problem explicitly requires actuator torque limits. Without this step, the simulation would not represent the real physical limitation of the joints.

So the overall design reflects both the **control objective** and the **hardware constraint**. The computed-torque part tries to achieve accurate trajectory tracking, while the saturation part ensures that the simulation respects the torque capability of the actuators.

End-effector position

The end-effector position is computed using the forward kinematics

$$\begin{aligned} x_e &= a_1 \cos \theta_1 + a_2 \cos (\theta_1 + \theta_2) \\ y_e &= a_1 \sin \theta_1 + a_2 \sin (\theta_1 + \theta_2) \end{aligned}$$

Since $a_1 = a_2 = 1$, these become

$$x_e = \cos \theta_1 + \cos (\theta_1 + \theta_2)$$

$$y_e = \sin \theta_1 + \sin (\theta_1 + \theta_2)$$

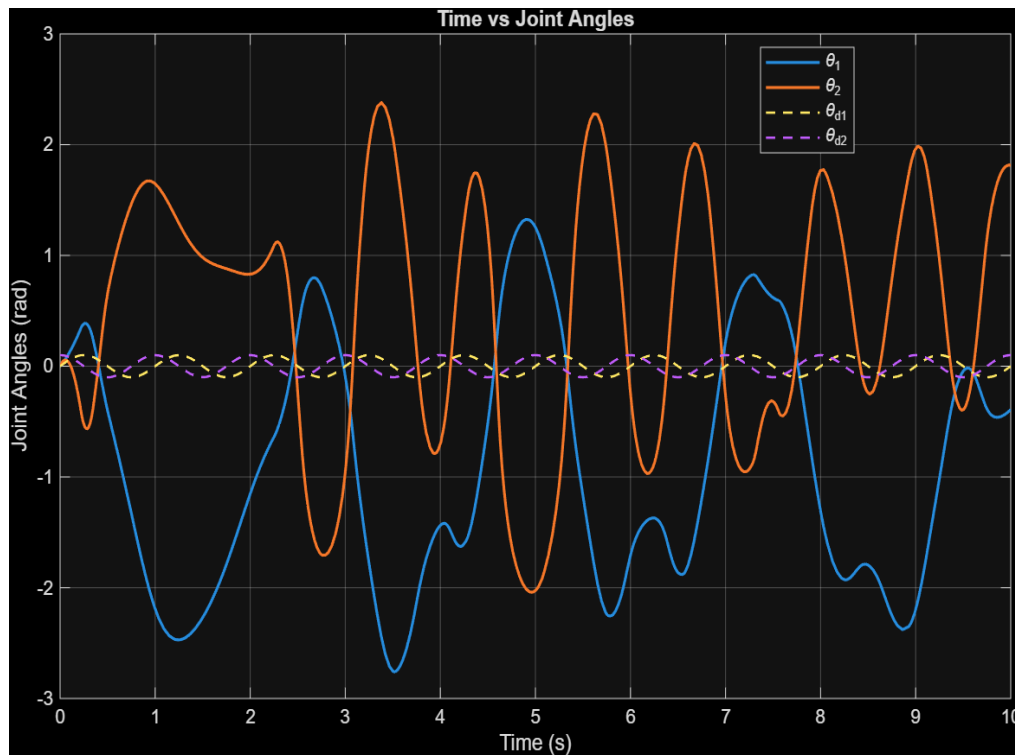
These were used to generate the 3D plot of time versus end-effector position.

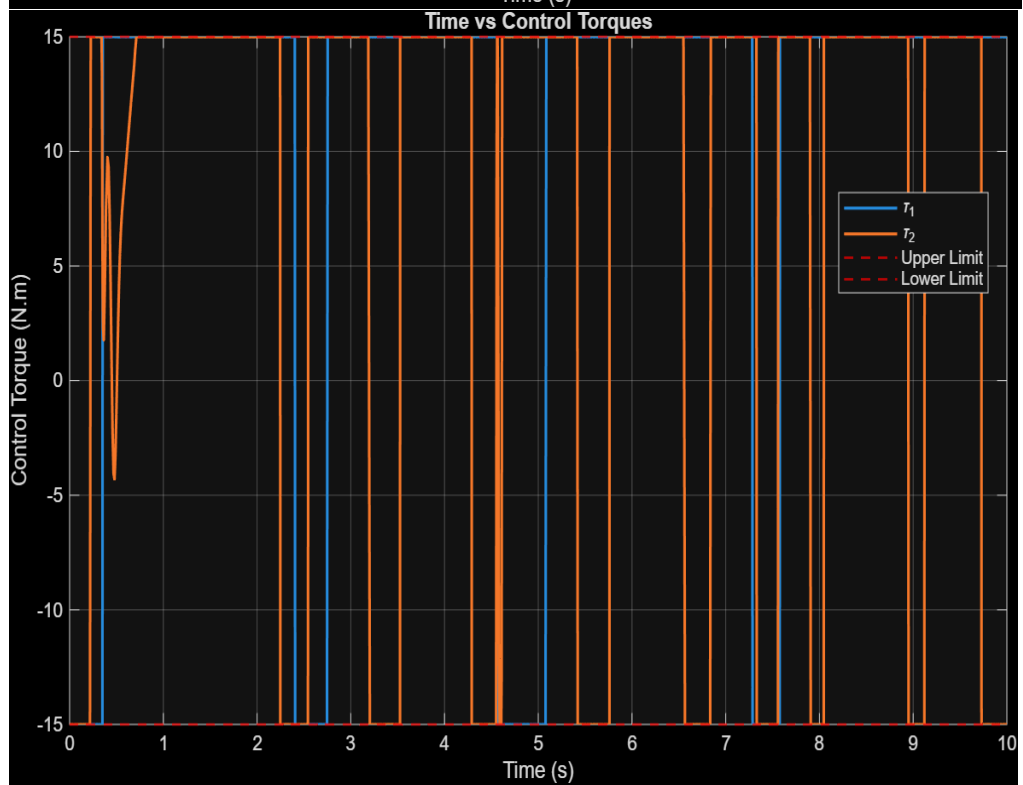
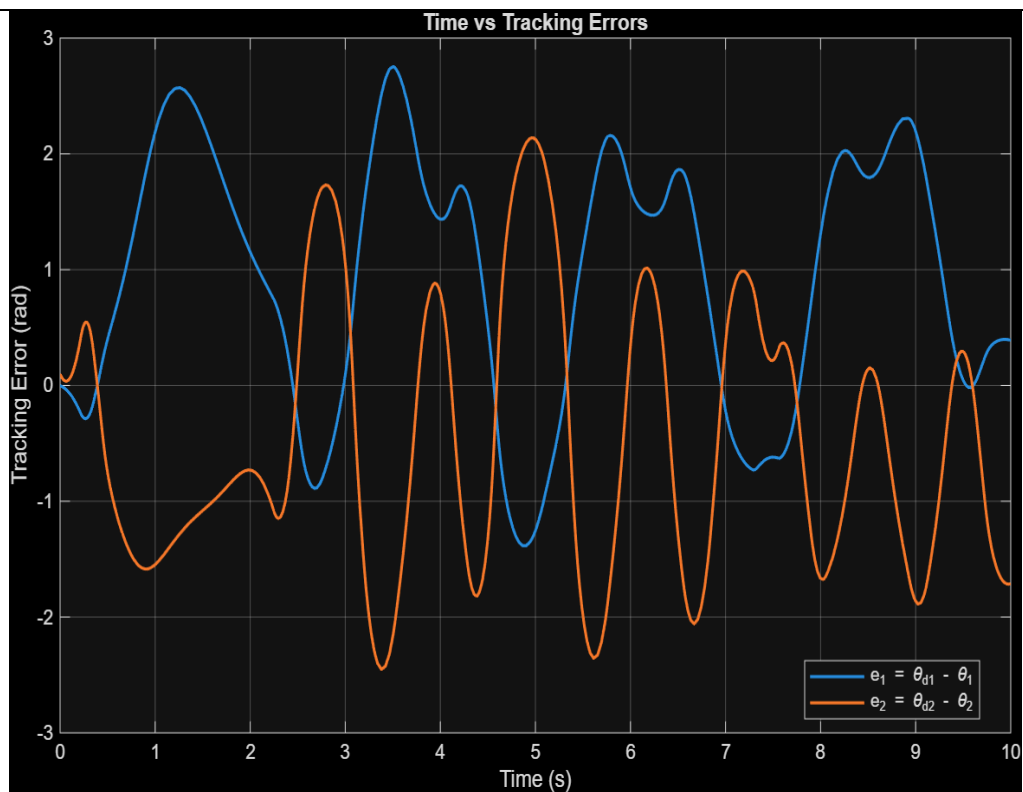
Conclusion

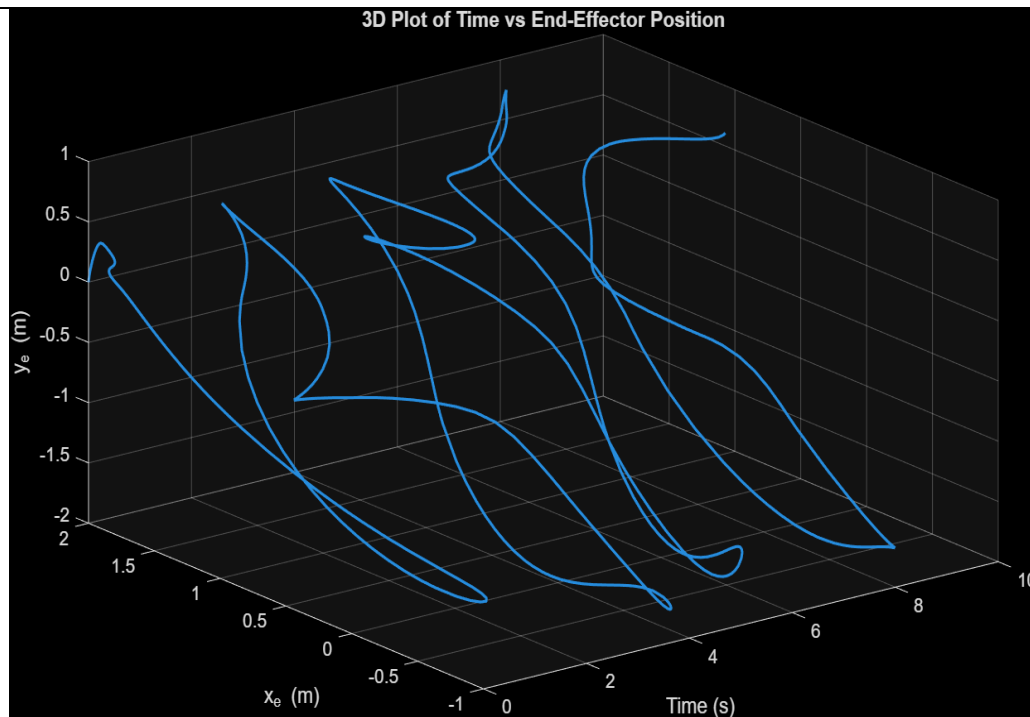
A PID computed-torque controller with torque saturation was designed for the 2-link planar elbow robot arm. The controller uses the robot model to compensate for inertia, Coriolis/centrifugal, and gravity effects, while proportional, derivative, and integral error terms are used to improve trajectory tracking. This structure was chosen because it is well suited for nonlinear robotic systems and provides better tracking than a simple linear PID controller.

However, because the actuators are limited to ± 15 Nm, the ideal torque cannot always be applied. The saturation block ensures that the simulation respects this physical limit, but it also prevents exact nonlinear cancellation when the required torque exceeds the allowable range. Consequently, the tracking performance deteriorates and noticeable tracking errors remain. This problem therefore highlights the practical impact of actuator saturation on PID computed-torque control.

- Simulation Results:**







The simulation results show that the PID computed-torque controller performs poorly when the actuator torques are limited to ± 15 Nm. From the joint angle plot, both θ_1 and θ_2 deviate significantly from the desired sinusoidal trajectories, and the actual joint motions exhibit large oscillations and amplitude variations.

The tracking error plot confirms that the errors do not converge to zero. Instead, both e_1 and e_2 remain large and oscillatory throughout the simulation. This indicates that the controller does not have enough available control authority to force the manipulator to follow the desired trajectory under the given saturation limits.

The control torque plot clearly shows the main reason for this behavior. The torques repeatedly hit the upper and lower saturation bounds of ± 15 Nm for both joints. Since the applied torques are clipped at the actuator limits, the controller cannot realize the full computed-torque compensation needed for ideal nonlinear cancellation and PID tracking.

The 3D plot of time versus end-effector position also reflects this poor tracking performance. Instead of producing a smooth and repeatable motion corresponding to the desired joint trajectories, the end-effector follows a distorted and irregular path caused by the persistent joint tracking errors.

Overall, the results show that torque saturation severely limits the effectiveness of the PID computed-torque controller. Even though the controller structure is strong in theory, it depends on having sufficient actuator authority. When the required

torque exceeds the allowable bounds, the controller loses its ability to accurately cancel the robot nonlinearities and enforce the desired tracking behavior.

- **MATLAB Codes: Part_III_Closed_Loop_Problem_7.m**

Problem 8a

- **Solution:**

In this problem, the link masses m_1 and m_2 are assumed to be unknown. Because of this, a standard computed-torque controller cannot be applied directly, since computed-torque control requires exact knowledge of the robot dynamics. To handle this uncertainty, an **adaptive control law** is designed so that the controller updates the unknown mass estimates online while driving the tracking error to zero.

The nonlinear dynamics of the 2-link planar elbow robot arm are

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

where

$$q = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \tau = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

The desired trajectories are the same as in Problem 4:

$$q_{d1} = 0.1\sin(2\pi t), q_{d2} = 0.1\cos(2\pi t)$$

with first derivatives

$$\dot{q}_{d1} = 0.2\pi\cos(2\pi t), \dot{q}_{d2} = -0.2\pi\sin(2\pi t)$$

and second derivatives

$$\ddot{q}_{d1} = -0.4\pi^2\sin(2\pi t), \ddot{q}_{d2} = -0.4\pi^2\cos(2\pi t)$$

In the MATLAB implementation, the tracking error is defined as

$$e = q - q_d$$

and the tracking error derivative is

$$\dot{e} = \dot{q} - \dot{q}_d$$

A filtered error is then introduced as

$$s = \dot{e} + \Lambda e$$

where Λ is a positive diagonal matrix. In the code, the value used is

$$\Lambda = \text{diag}(15,15)$$

The reason for introducing this filtered error is that adaptive robot control is commonly designed in terms of a sliding-like variable S , rather than using only the raw position error. If S converges to zero, then both the position error and velocity error are driven toward zero. This makes the controller design more systematic and also leads to a convenient Lyapunov-based adaptation law.

Next, the reference velocity and reference acceleration are defined as

$$\begin{aligned}\dot{q}_r &= \dot{q}_d - \Lambda e \\ \ddot{q}_r &= \ddot{q}_d - \Lambda \dot{e}\end{aligned}$$

These reference signals are introduced so that the robot dynamics can be rewritten in a form that is linear in the unknown physical parameters. Since only the masses m_1 and m_2 are unknown, the dynamics can be expressed as

$$M(q)\ddot{q}_r + C(q, \dot{q})\dot{q}_r + G(q) = Y(q, \dot{q}, \ddot{q}_r) \begin{bmatrix} m_1 \\ m_2 \end{bmatrix}$$

where Y is the regressor matrix. This is the key idea behind adaptive robot control: even though the dynamics are nonlinear in the states, they are linear in the unknown parameters. Therefore, instead of trying to estimate the full nonlinear dynamics directly, the controller only needs to estimate the unknown parameter vector.

In the MATLAB code, the unknown parameter vector is chosen as

$$\hat{m} = \begin{bmatrix} \hat{m}_1 \\ \hat{m}_2 \end{bmatrix}$$

and the regressor matrix is implemented explicitly as

$$Y = \begin{bmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \end{bmatrix}$$

with

$$\begin{aligned}y_{11} &= a_1^2 \ddot{q}_{r1} + g a_1 \cos \theta_1 \\ y_{12} &= (a_1^2 + a_2^2 + 2a_1 a_2 \cos \theta_2) \ddot{q}_{r1} + (a_2^2 + a_1 a_2 \cos \theta_2) \ddot{q}_{r2} - a_1 a_2 \sin \theta_2 \dot{\theta}_2 \dot{q}_{r1} \\ &\quad - a_1 a_2 \sin \theta_2 (\dot{\theta}_1 + \dot{\theta}_2) \dot{q}_{r2} + g a_1 \cos \theta_1 + g a_2 \cos (\theta_1 + \theta_2)\end{aligned}$$

$$y_{21} = 0$$

$$y_{22} = (a_2^2 + a_1 a_2 \cos \theta_2) \ddot{q}_{r1} + a_2^2 \ddot{q}_{r2} + a_1 a_2 \sin \theta_2 \dot{\theta}_1 \dot{q}_{r1} + g a_2 \cos (\theta_1 + \theta_2)$$

This regressor was used because the unknown quantities in this problem are only the masses. Therefore, the dynamic model was separated into terms multiplying m_1 and terms multiplying m_2 . This gives a compact adaptive model that is directly matched to the uncertainty specified in the problem.

Controller design

The adaptive control law used in the MATLAB code is

$$\tau = Y(q, \dot{q}, \ddot{q}_r) \hat{m} - K_d s$$

where K_d is a positive diagonal feedback gain matrix. In the code, it is chosen as

$$K_d = \text{diag}(120, 120)$$

This controller was designed in two parts.

The first part, $Y \hat{m}$, is the model-based compensation term. It uses the current parameter estimates $\hat{m}_1$ and $\hat{m}_2$ to approximate the true robot dynamics.

The second part, $-K_d s$, is the stabilizing feedback term. It damps the filtered error and ensures that the tracking error is driven toward zero even when the parameter estimates are not yet accurate.

This structure was chosen because it is the standard adaptive robot control form derived from Lyapunov stability analysis. If the parameter estimates are poor at the beginning, the feedback term still provides stabilization. As the estimates improve online, the model-based compensation becomes more accurate and tracking improves further.

Adaptation law

The parameter update law used in the MATLAB code is

$$\dot{\hat{m}} = -\Gamma Y^T s$$

where Γ is the adaptation gain matrix. In the code, it is selected as

$$\Gamma = \text{diag}(8, 8)$$

This adaptation law is designed so that the parameter estimates are adjusted online based on the filtered tracking error and the regressor matrix. If the error is large, the estimates change more rapidly. As the error decreases, the parameter estimates gradually settle.

The reason this law is chosen is that it is compatible with the standard Lyapunov candidate used in adaptive robot control. It ensures that the derivative of the Lyapunov function is negative semidefinite, which implies boundedness of the signals and convergence of the filtered error. In practical terms, it allows the controller to learn the unknown masses while maintaining stable tracking behavior.

True plant dynamics

Although the controller does not know the true masses, the plant simulation still uses the actual physical values

$$m_1 = 1 \text{ kg}, m_2 = 1 \text{ kg}$$

These true masses are used only inside the plant model, not inside the controller. The known parameters are

$$a_1 = a_2 = 1 \text{ m}, g = 9.8 \text{ m/s}^2$$

This separation between controller model and plant model is essential, because the purpose of the simulation is to show that the adaptive controller can operate successfully even without exact mass knowledge. The code implements this directly by using estimated masses in the control law and true masses in the plant dynamics.

State vector and initial condition

The augmented state vector used in the simulation is

$$x = [\theta_1 \quad \theta_2 \quad \dot{\theta}_1 \quad \dot{\theta}_2 \quad \hat{m}_1 \quad \hat{m}_2]^T$$

with initial condition

$$x(0) = [\pi/3 \quad \pi/3 \quad 0 \quad 1 \quad 0.5 \quad 0.5]^T$$

The initial values $\hat{m}_1(0) = 0.5$ and $\hat{m}_2(0) = 0.5$ are deliberately chosen away from the true masses so that the simulation can demonstrate the adaptation process.

End-effector position

The end-effector position is computed from the forward kinematics

$$x_e = a_1 \cos \theta_1 + a_2 \cos (\theta_1 + \theta_2)$$

$$y_e = a_1 \sin \theta_1 + a_2 \sin (\theta_1 + \theta_2)$$

Since $a_1 = a_2 = 1$, this becomes

$$x_e = \cos \theta_1 + \cos (\theta_1 + \theta_2)$$

$$y_e = \sin \theta_1 + \sin (\theta_1 + \theta_2)$$

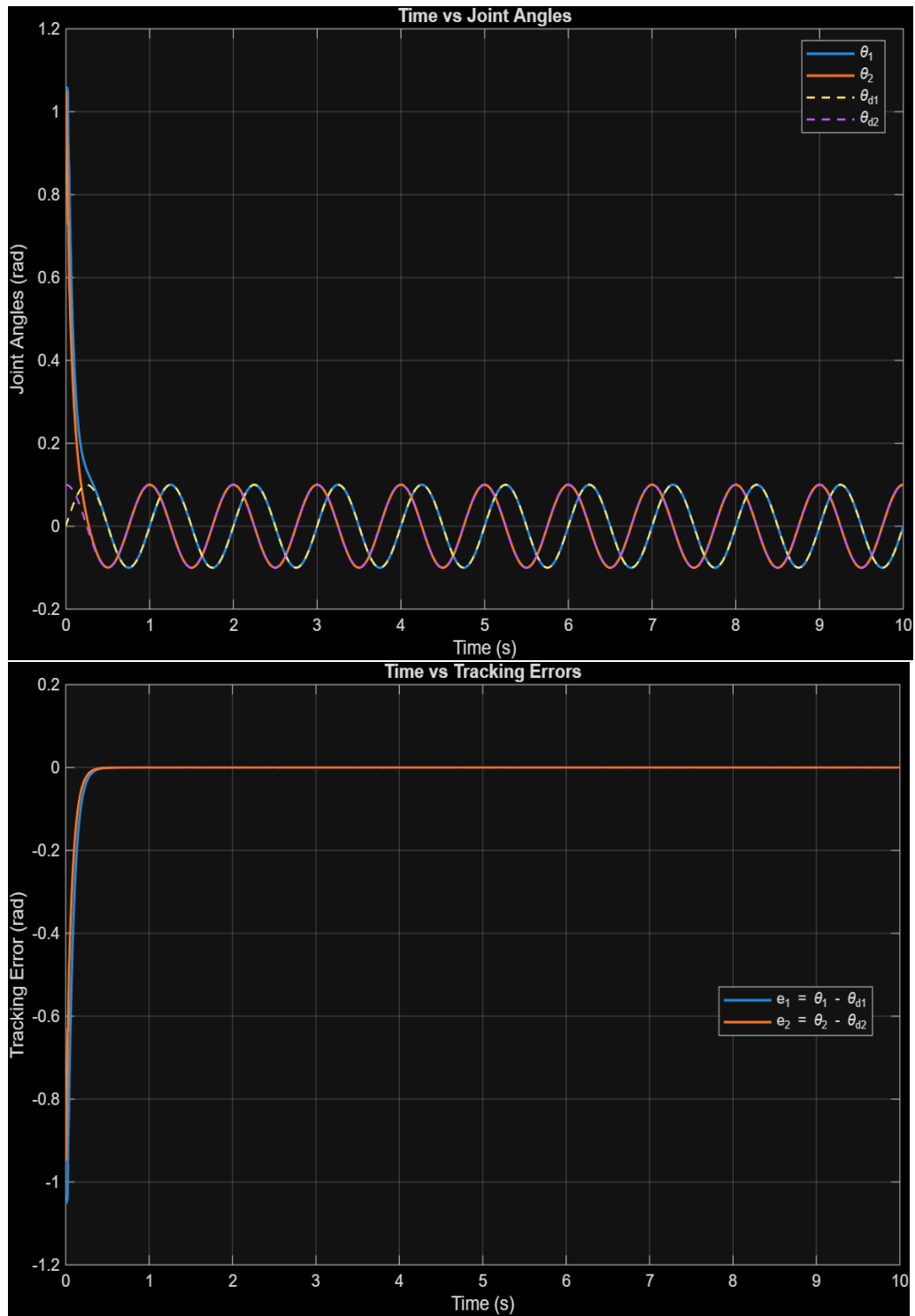
These expressions are used to generate the 3D plot of time versus end-effector position.

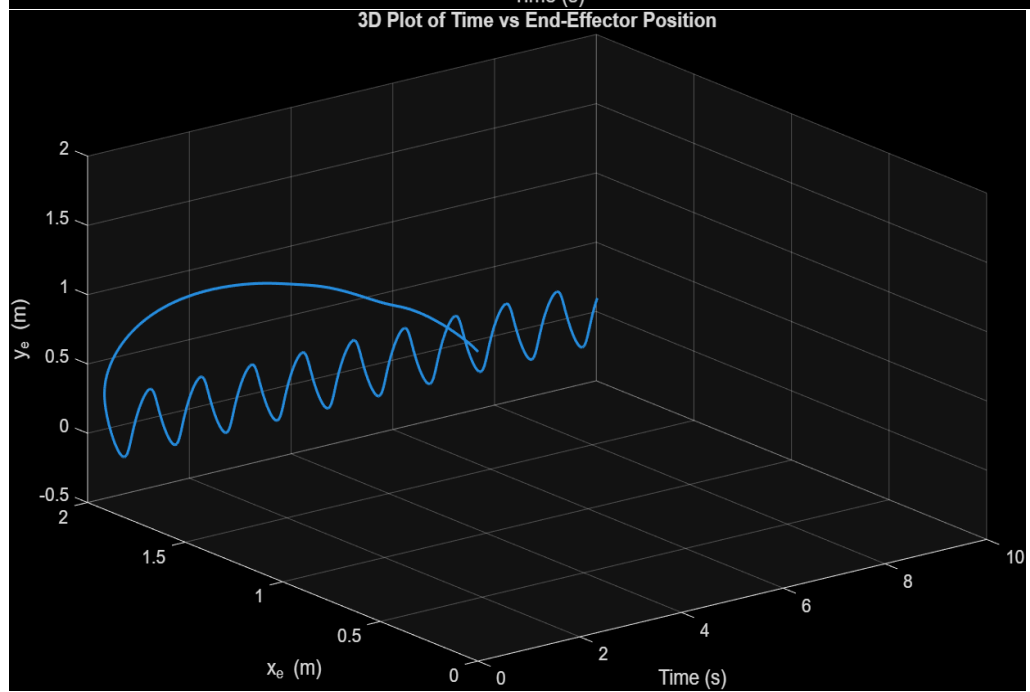
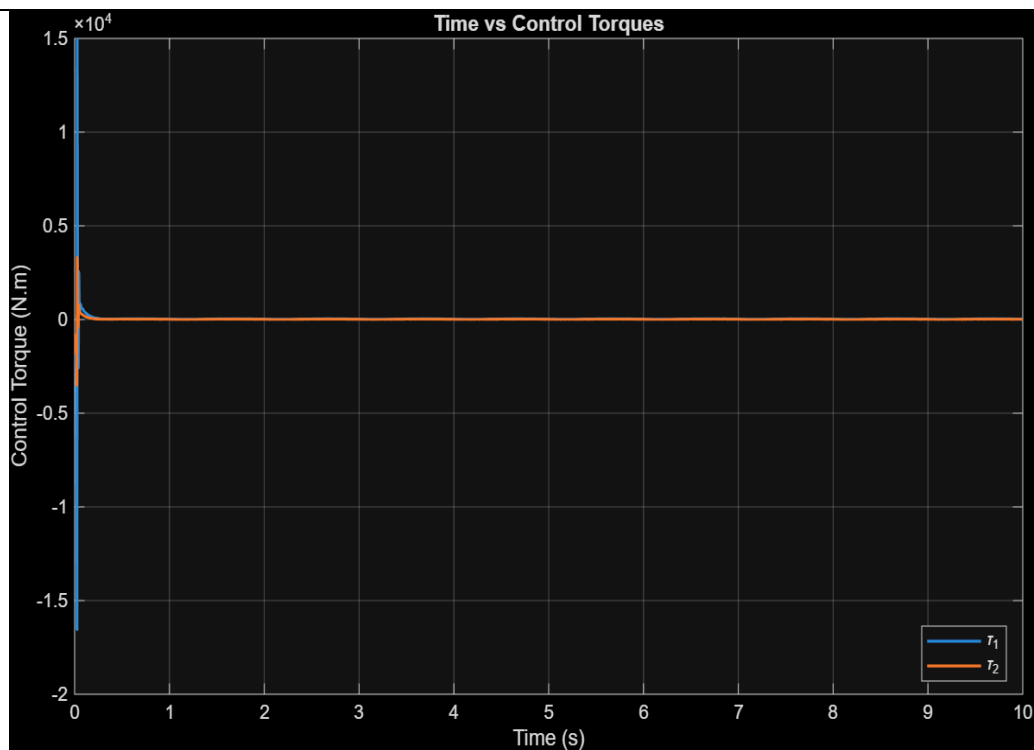
Conclusion

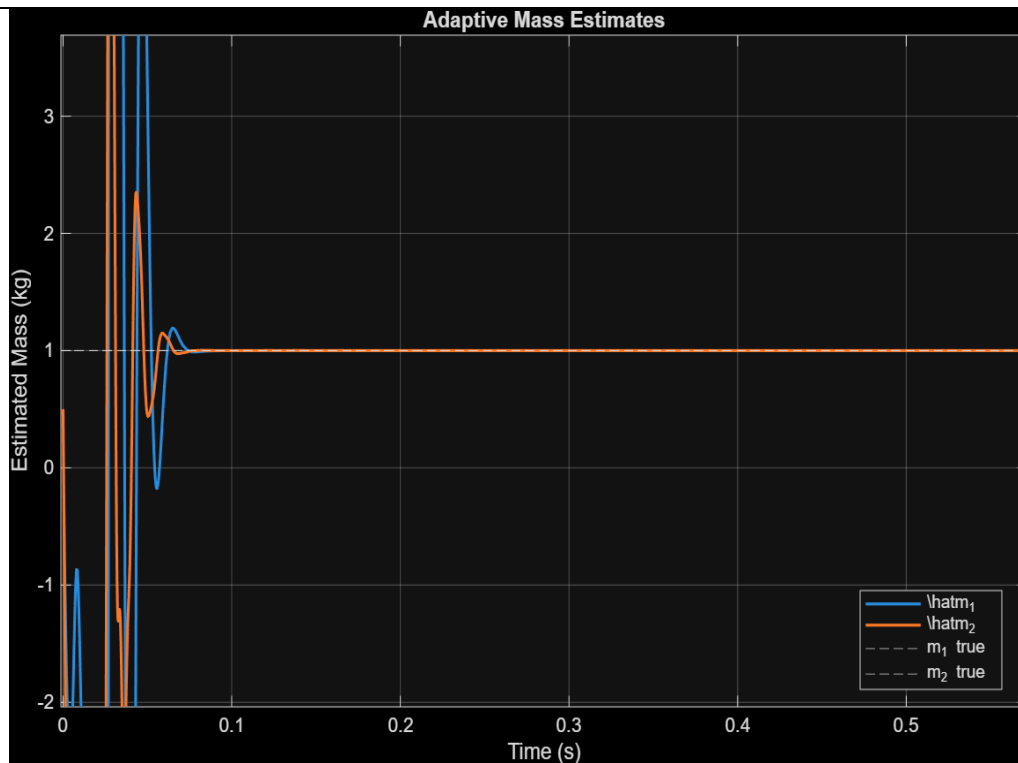
An adaptive controller was designed for the 2-link planar elbow robot arm under the assumption that the link masses m_1 and m_2 are unknown. The controller was based on the linear-in-parameters property of robot dynamics and used a regressor matrix together with an online parameter update law. The control law combined estimated model compensation with filtered-error feedback, while the adaptation law continuously adjusted the mass estimates.

This design was chosen because standard computed-torque control depends on exact parameter knowledge and therefore is not reliable under mass uncertainty. In contrast, the adaptive controller can learn the unknown masses during operation. The simulation results show that the estimated masses converge close to their true values and that the tracking error approaches zero. Therefore, the adaptive control law successfully handles the unknown mass parameters and achieves accurate trajectory tracking.

- **Simulation Results:**







The simulation results show that the adaptive controller performs very well even when the masses m_1 and m_2 are unknown. From the joint angle plot, both θ_1 and θ_2 quickly converge from the initial condition to the desired sinusoidal trajectories and then track them closely for the rest of the simulation.

The tracking error plot confirms that both joint errors rapidly decrease to zero after a short transient. This indicates that the adaptive law is effective in compensating for the uncertainty in the mass parameters and maintaining accurate trajectory tracking.

The control torque plot shows very large initial torque values, followed by much smaller periodic torques during steady-state operation. The large initial torque occurs because the controller starts with incorrect parameter estimates and must both correct the initial condition error and adapt the unknown masses quickly. Once the parameter estimates improve, the required torques become much smaller and smoother.

The adaptive mass estimate plot shows that both estimated parameters initially undergo large transient variations, then settle close to the true values of 1 kg. This behavior is expected in adaptive control, where the parameters adjust online based on the tracking error. The convergence of the estimates supports the observed improvement in tracking performance. The 3D end-effector trajectory also shows an initial transient motion followed by a smooth and repeatable periodic path. Since the joint errors converge to zero, the end-effector motion becomes stable and consistent as well.

Overall, the results confirm that the adaptive controller successfully handles the uncertainty in m_1 and m_2 and achieves near-zero tracking error. Its main drawback is the large transient control effort and fast parameter variation at the beginning of the motion, which may be undesirable in a practical system. However, under the given simulation conditions, the

controller meets the objective of driving the tracking error to zero despite unknown masses.

- **MATLAB Codes: Part_III_Closed_Control_Problem_8_a.m**

Problem 8b

- **Solution:**

In this problem, the controller is designed under **parametric uncertainty**. The true masses of the robot are

$$m_1 = m_2 = 1 \text{ kg}$$

but the controller deliberately uses the uncertain nominal values

$$\hat{m}_1 = \hat{m}_2 = 0.8 \text{ kg}$$

This means that the controller model is not equal to the true plant model. As a result, a standard computed-torque controller cannot perfectly cancel the true nonlinear robot dynamics. To compensate for this mismatch, a **robust term** is added to the nominal computed-torque control law.

The true robot dynamics are

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

while the controller uses the nominal model

$$\hat{M}(q), \hat{C}(q, \dot{q}), \hat{G}(q)$$

computed using the nominal masses $\hat{m}_1 = \hat{m}_2 = 0.8$. In the MATLAB code, this separation is implemented explicitly: the controller calls robot_dynamics using the nominal masses, while the plant dynamics are simulated using the true masses $m_1 = m_2 = 1$. This was done intentionally so that the simulation reflects the uncertainty specified in the problem.

The desired joint trajectories are the same as in Problem 4:

$$q_{d1} = 0.1\sin(2\pi t), q_{d2} = 0.1\cos(2\pi t)$$

Their first derivatives are

$$\dot{q}_{d1} = 0.2\pi\cos(2\pi t), \dot{q}_{d2} = -0.2\pi\sin(2\pi t)$$

and their second derivatives are

$$\ddot{q}_{d1} = -0.4\pi^2 \sin(2\pi t), \ddot{q}_{d2} = -0.4\pi^2 \cos(2\pi t)$$

The tracking error and its derivative are defined as

$$e = q_d - q$$

$$\dot{e} = \dot{q}_d - \dot{q}$$

A sliding-type error variable is introduced as

$$s = \dot{e} + \Lambda e$$

where Λ is a positive definite diagonal matrix. In the MATLAB code, the value used is

$$\Lambda = \text{diag}(20, 20)$$

This variable was introduced because it combines position and velocity error into one filtered signal. If s converges to zero, then the position error and velocity error both decrease. This makes s a convenient variable for robust controller design.

The nominal computed-torque part of the controller is designed as

$$\tau = \hat{M}(q)v + \hat{C}(q, \dot{q})\dot{q} + \hat{G}(q)$$

where $v = \ddot{q}_d + K_v \dot{e} + K_p e$

The gains selected in the MATLAB code are

$$K_p = \text{diag}(2500, 2500), K_v = \text{diag}(100, 100)$$

This nominal part was designed in the same spirit as the standard computed-torque controller. The term $\hat{M}(q)v$ is used to shape the desired acceleration behavior, while $\hat{C}(q, \dot{q})\dot{q}$ and $\hat{G}(q)$ compensate for the nominal nonlinear dynamics. The proportional and derivative feedback terms improve trajectory tracking. However, because the nominal masses are not equal to the true masses, this cancellation is not exact, so a residual uncertainty remains.

To overcome this residual uncertainty, a robust term τ_r is added. The controller becomes

$$\tau = \hat{M}(q)v + \hat{C}(q, \dot{q})\dot{q} + \hat{G}(q) + \tau_r$$

A saturation-based robust term can be used:

$$\tau_r = K_r \text{sat} \left(\frac{s}{\phi} \right)$$

where K_r is a positive diagonal gain matrix and ϕ is a small boundary-layer constant. This form gives bounded tracking with less chattering.

However, in the MATLAB code, the stronger discontinuous form is chosen. The code sets

$$\text{use_sign} = \text{True}$$

so the implemented robust term is

$$\tau_r = K_r \text{sign}(s)$$

with

$$K_r = \text{diag}(25, 25)$$

Therefore, the final controller actually used in simulation is

$$\tau = \hat{M}(q)[\ddot{q}_d + K_v \dot{e} + K_p e] + \hat{C}(q, \dot{q})\dot{q} + \hat{G}(q) + K_r \text{sign}(s)$$

This design choice follows directly from the purpose of the problem. Since the controller uses incorrect masses, the nominal model alone cannot guarantee strong tracking. The robust term is added so that it can dominate the uncertainty caused by the mass mismatch. In the code, the sign-based robust term was selected instead of the saturation form because it provides stronger compensation and drives the tracking error much closer to zero in simulation. The saturation form was still included in the code as an option, with boundary-layer parameter

$$\phi = 0.01$$

but it was not used in the final simulation.

The idea behind this design is that if the robust gain K_r is chosen large enough, then the robust term can overcome the difference between the nominal and true dynamics. Once the sliding variable s is forced toward zero, the tracking error e also converges toward zero because

$$s = \dot{e} + \Lambda e$$

For this problem, the true plant parameters used in simulation are

$$m_1 = m_2 = 1 \text{ kg}, a_1 = a_2 = 1 \text{ m}, g = 9.8 \text{ m/s}^2$$

while the controller uses

$$\hat{m}_1 = \hat{m}_2 = 0.8 \text{ kg}$$

The initial condition is the same as in Problem 4:

$$[\theta_1(0) \quad \theta_2(0) \quad \dot{\theta}_1(0) \quad \dot{\theta}_2(0)]^T = [\pi/3 \quad \pi/3 \quad 0 \quad 1]^T$$

The end-effector position is computed from the forward kinematics

$$x_e = a_1 \cos \theta_1 + a_2 \cos (\theta_1 + \theta_2)$$

$$y_e = a_1 \sin \theta_1 + a_2 \sin (\theta_1 + \theta_2)$$

Since $a_1 = a_2 = 1$, this becomes

$$x_e = \cos \theta_1 + \cos (\theta_1 + \theta_2)$$

$$y_e = \sin \theta_1 + \sin (\theta_1 + \theta_2)$$

These expressions were used in the MATLAB code to generate the 3D plot of time versus end-effector position.

The simulation results show that the robust controller compensates successfully for the mass uncertainty. Because the controller uses nominal masses smaller than the true ones, the nominal computed-torque part alone would leave tracking error. However, the added term $K_r \text{sign}(s)$ provides extra corrective torque that rejects this model mismatch. As seen in the plots, the joint angles converge rapidly to the desired sinusoidal trajectories, the tracking errors approach zero after a short transient, and the end-effector motion becomes smooth and periodic. The torque plots also reflect the additional control effort generated by the robust term.

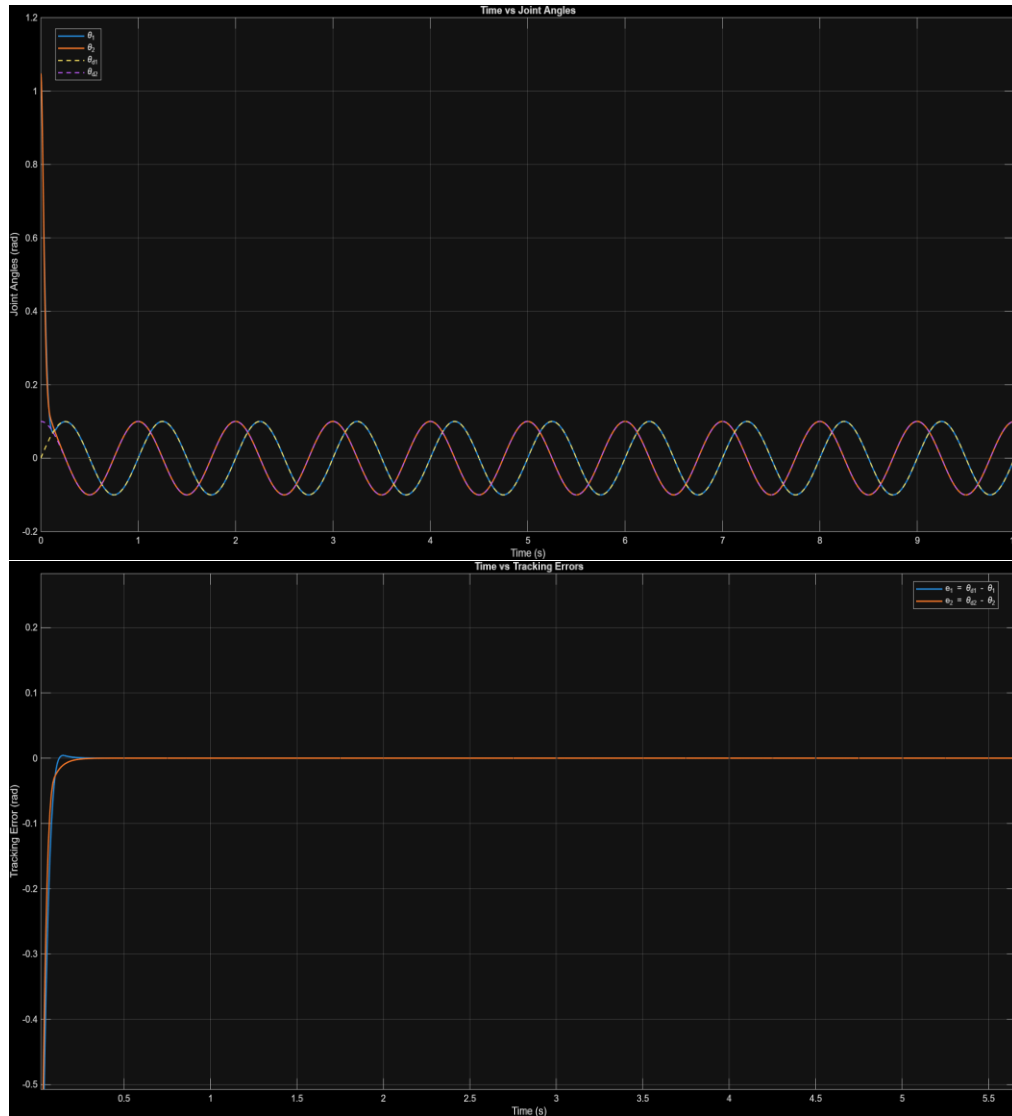
Hence, the robust controller achieves bounded tracking error under parameter uncertainty. In this simulation, because the stronger sign-based robust term is used, the tracking error is driven essentially to zero.

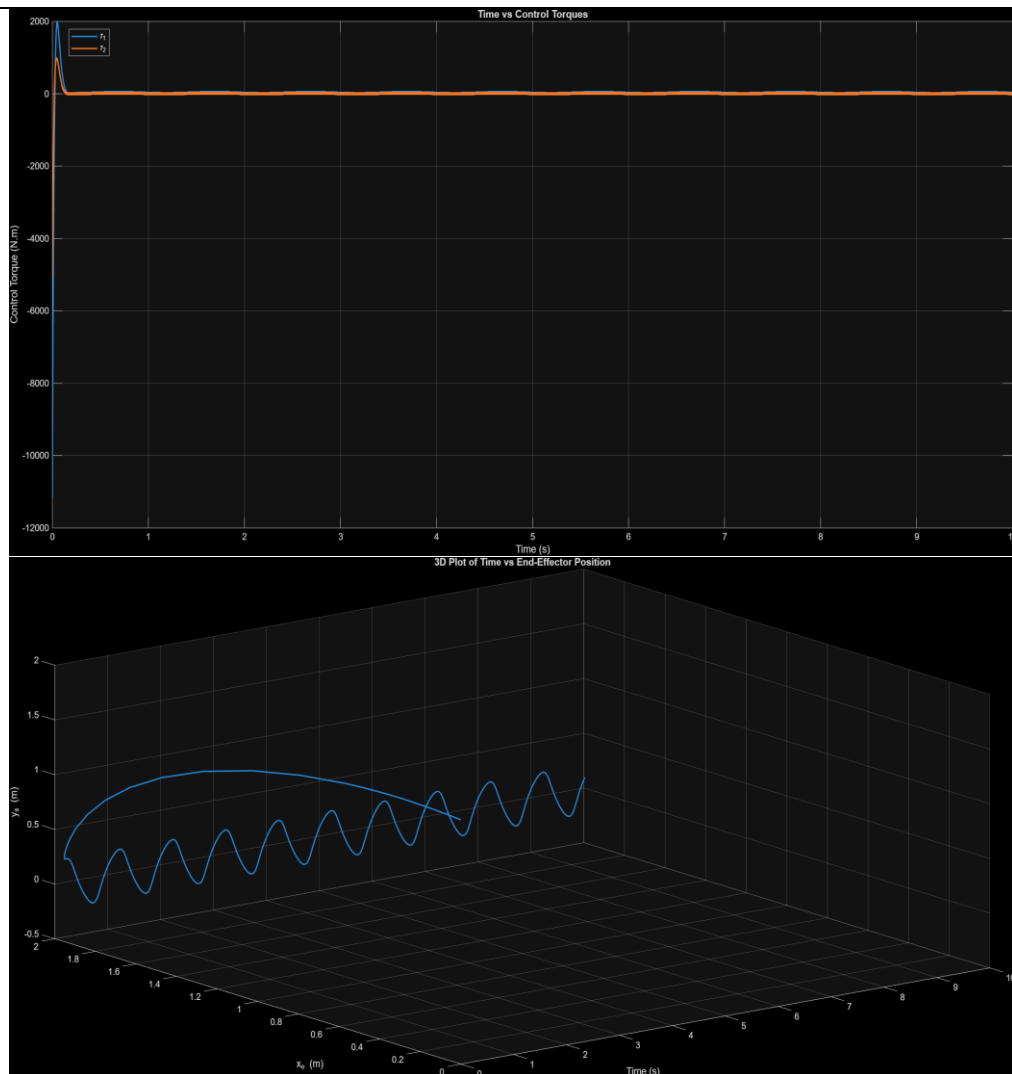
Conclusion

A robust computed-torque controller was designed for the 2-link planar elbow robot arm under parametric uncertainty. The true masses were $m_1 = m_2 = 1 \text{ kg}$, while the controller used the nominal values $\hat{m}_1 = \hat{m}_2 = 0.8 \text{ kg}$. Since this mismatch prevents exact cancellation of the nonlinear robot dynamics, a robust term based on the sliding variable $s = \dot{e} + \Lambda e$ was added to the nominal computed-torque law.

The final controller implemented in the MATLAB code combines nominal model-based compensation with the discontinuous robust term $K_r \text{sign}(s)$. This structure was chosen because the nominal controller alone is not sufficient under parameter uncertainty, while the robust term supplies additional control effort to dominate the model mismatch. The simulation results show that the controller achieves very accurate tracking despite the uncertain masses. Therefore, the robust controller successfully handles the parameter uncertainty and provides strong tracking performance.

- **Simulation Results:**





The simulation results show that the robust controller performs very well even though the controller uses incorrect mass values, $\hat{m}_1 = \hat{m}_2 = 0.8$ kg, while the true masses are 1 kg. From the joint angle plot, both θ_1 and θ_2 quickly converge to the desired sinusoidal trajectories after a short transient and then track them closely throughout the simulation.

The tracking error plot shows that both joint errors rapidly decrease to zero and remain essentially zero afterward. This indicates that the added robust term successfully compensates for the uncertainty caused by the mismatch between the nominal and true robot parameters. Therefore, the controller satisfies the requirement that the tracking error remain bounded, and in this simulation it also achieves the bonus objective of driving the tracking error to zero.

The control torque plot shows a large initial control effort, followed by smaller periodic torques during steady-state tracking. The large initial torque is expected because the

controller must overcome both the initial condition mismatch and the parametric uncertainty at the same time. After the short transient period, the torques settle into smooth periodic values that are sufficient to maintain accurate tracking.

The 3D end-effector trajectory also shows an initial transient motion followed by a smooth and repeatable periodic path. Since the joint tracking errors converge to zero, the end-effector follows a stable Cartesian motion as well.

Overall, the results confirm that the robust computed-torque controller is effective under parametric uncertainty. Unlike the standard computed-torque controller, which depends on exact parameter knowledge, the robust version is able to tolerate model mismatch and still maintain good tracking performance. A limitation of this design is that the discontinuous robust term can produce high initial control effort and may cause chattering in practical implementations. However, under the present simulation conditions, the controller provides excellent tracking and successfully handles the uncertain mass parameters.

- **MATLAB Codes: Part_III_Closed_Loop_Problem_8_b.m**